

平衡树

Hex

1 FHQ Treap 简介



为什么需要平衡树？

普通 BST 在有序插入时会退化成链，时间复杂度退化为 $O(n)$ 。



Treap 的思想

每个节点附带一个随机优先级 priority，同时满足 BST 性质与堆性质，树高期望为 $O(\log n)$ 。



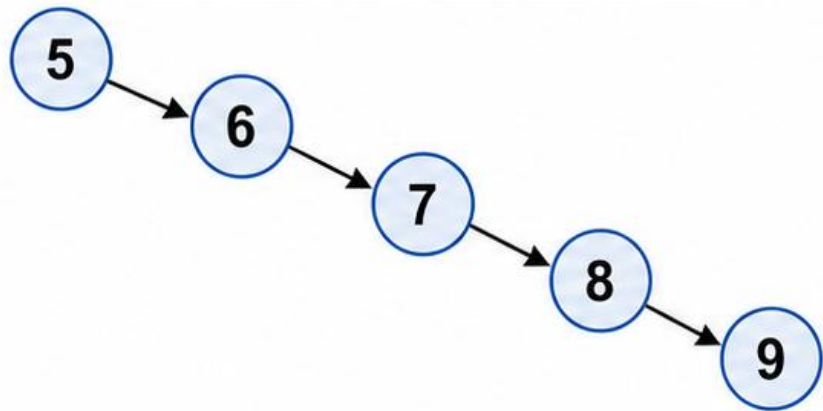
FHQ Treap 特点

- ✓ 无旋转 (Rotation)
- ✓ 所有操作基于 Split 和 Merge
- ✓ 代码简洁，易于实现
- ✓ 易于扩展 (可持久化、区间操作等)

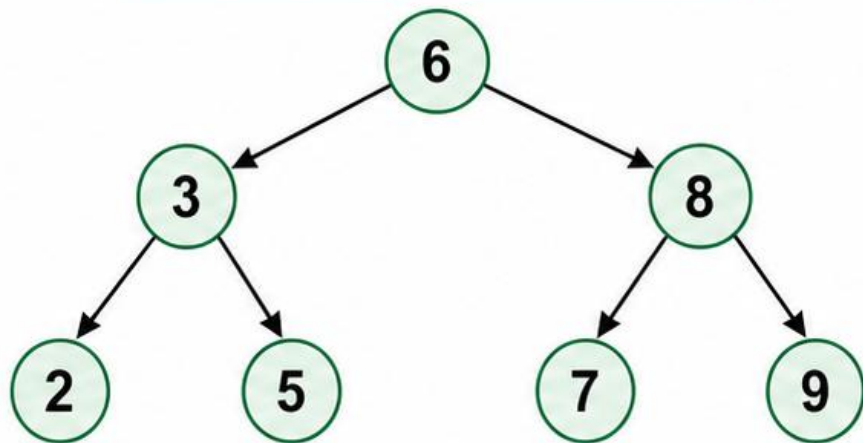


FHQ Treap = BST + 随机优先级 + Split / Merge

BST 退化 (有序插入)



FHQ Treap (平衡形态)

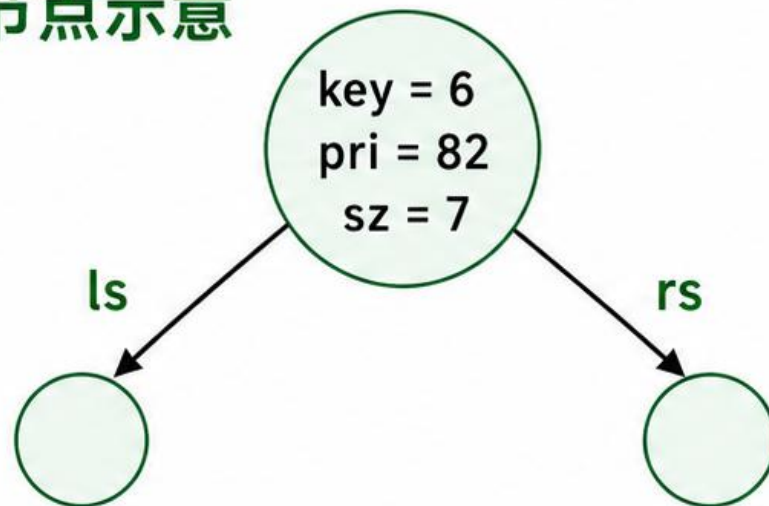


2 节点维护

</> 节点结构与更新函数

```
struct Node{  
    int ls, rs, key, pri, sz;  
} t[Maxn];  
  
void update(int u){  
    t[u].sz = t[t[u].ls].sz +  
             t[t[u].rs].sz + 1;  
}
```

⚙ 节点示意



- **key** = BST关键字 (决定中序有序)
- **pri** = 随机优先级 (决定树形)
- **ls/rs** = 左右儿子
- **sz** = 以该点为根的子树大小



维护好 **sz**，才能支持排名、第 **k** 小等操作。

3

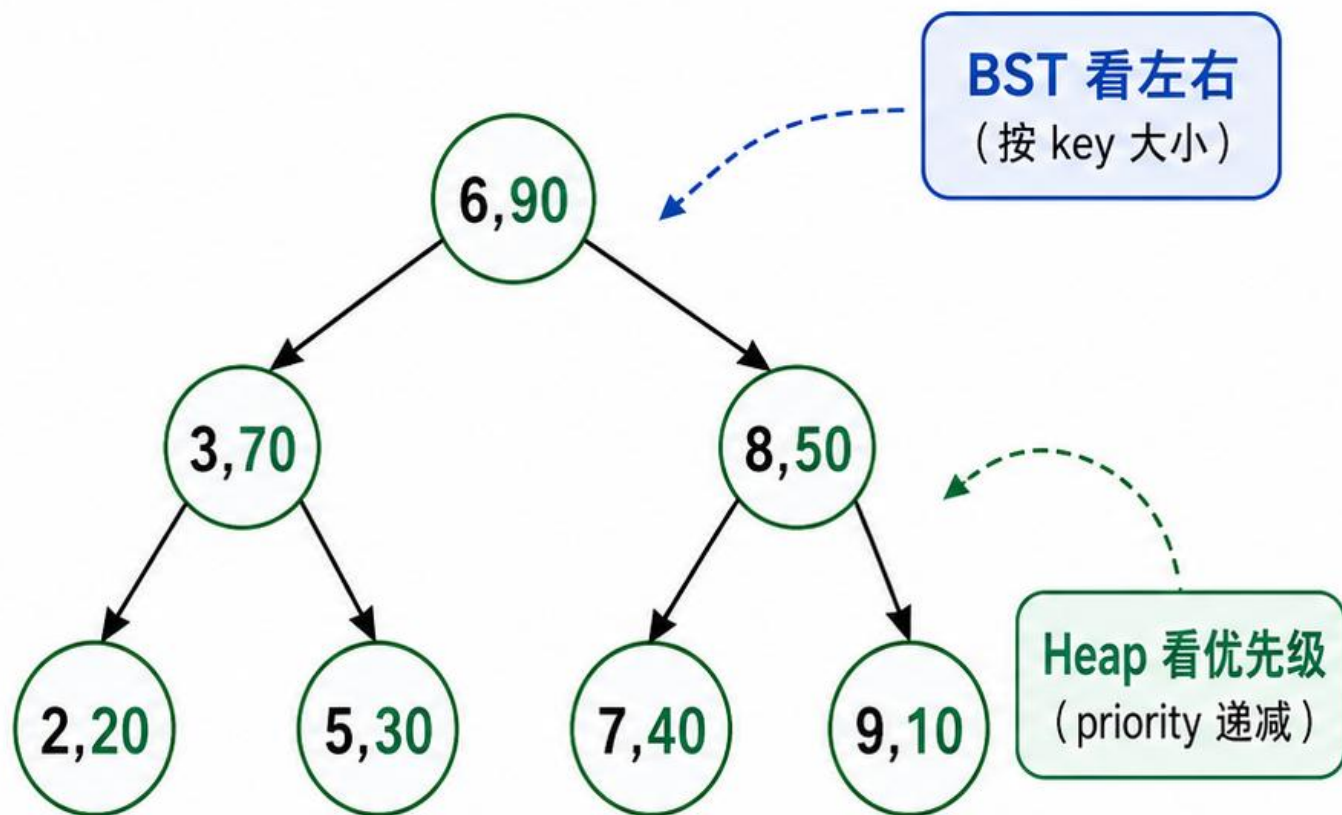
Treap 满足两个性质

BST 性质 (按 key)

左子树所有 **key** < 根 **key** < 右子树所有 **key**。

堆 (Heap) 性质 (按 pri)

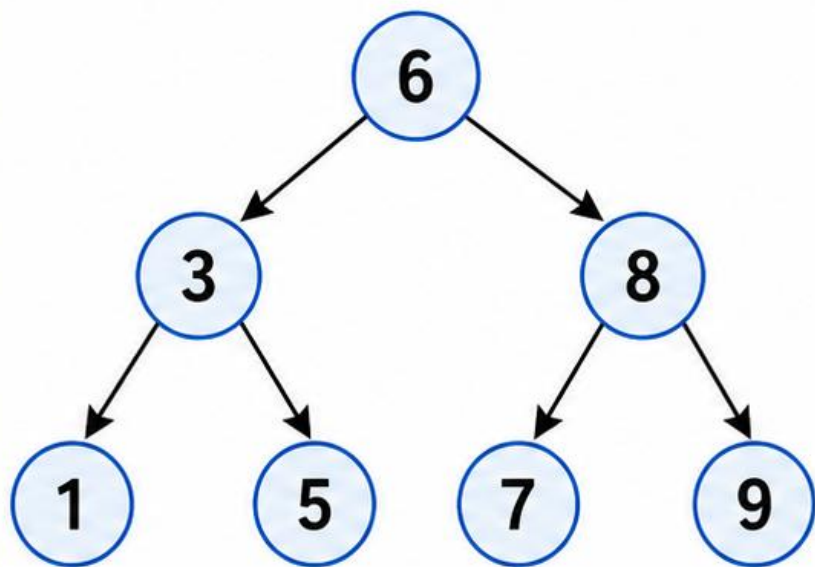
父节点的 **priority** 大于子节点的 **priority**
(与示例代码 merge 条件一致)。



BST 决定查找方向, **priority** 决定树形, 二者结合保证树高期望为 $O(\log n)$ 。

4 Split 思想 (重点)

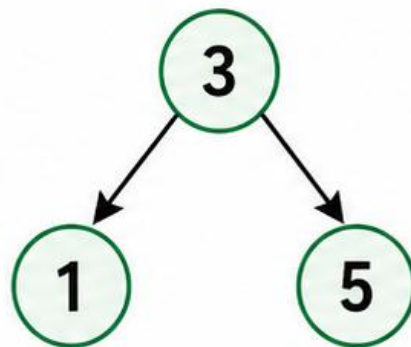
原始树



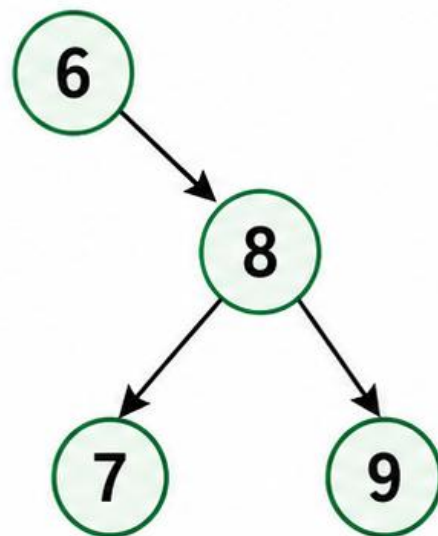
split(root, 5)



左树 L: ≤ 5



右树 R: > 5



★ Split 将一棵树按 **key** 分成两棵树: L 中所有 **key** $\leq x$, R 中所有 **key** $> x$ 。

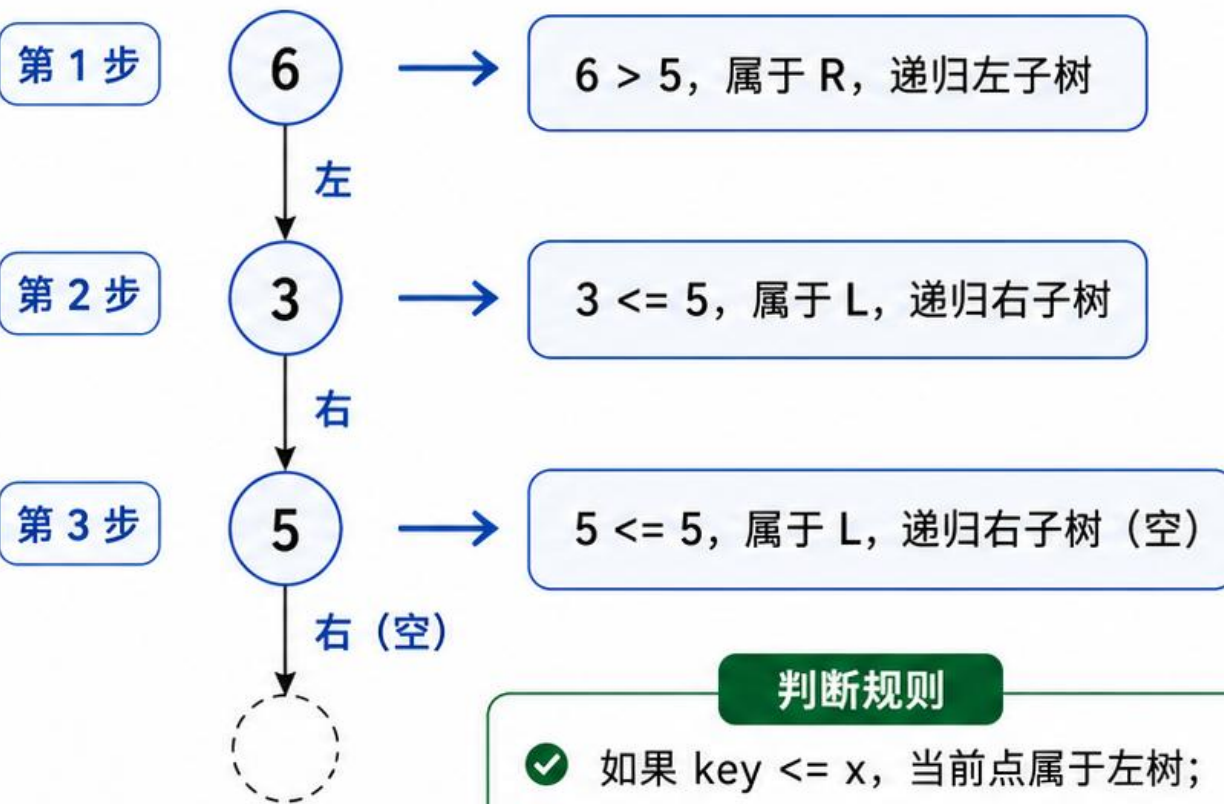
5 Split 递归过程 (重点)

split 函数代码

```
void split(int u, int x, int &L, int &R){  
    if(u==0){  
        L=R=0;  
        return;  
    }  
    if(t[u].key <= x){  
        L = u;  
        split(t[u].rs, x, t[u].rs, R);  
    } else {  
        R = u;  
        split(t[u].ls, x, L, t[u].ls);  
    }  
    update(u);  
}
```

★ 调用：split(root, 5, L, R)

以 split(root, 5) 为例的递归过程



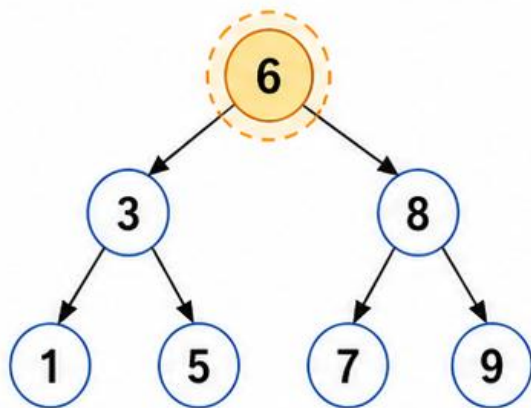
💡 本质：不断决定当前节点属于 L 还是 R, 并递归处理另一侧儿子。

6 Split 动态过程 (重点)

? 示例: `split(root, 5)` 将以 6 为根的 Treap 按值 5 进行 Split, 分成两个子树 L (≤ 5) 和 R (> 5)。

Step 1

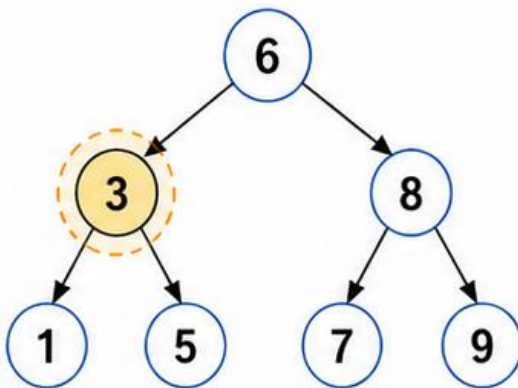
比较根节点 6 与 5:
 $6 > 5$, 放入右树 R



$6 > 5$, 根节点放入右树 R,
对左子树 3 继续 `split(3, 5)`。

Step 2

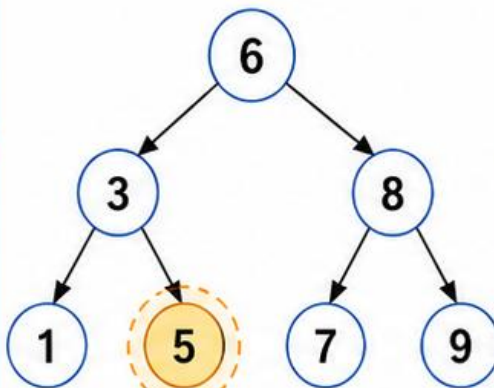
比较节点 3 与 5:
 $3 \leq 5$, 放入左树 L



$3 \leq 5$, 节点 3 放入左树 L,
对右子树 5 继续 `split(5, 5)`。

Step 3

比较节点 5 与 5:
 $5 \leq 5$, 放入左树 L

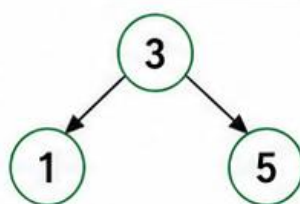


$5 \leq 5$, 节点 5 放入左树 L,
其左、右子树均为空, 返回。

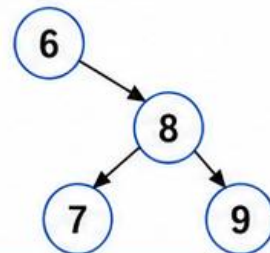
最终结果

Split 完成, 得到两个子树:

L: ≤ 5

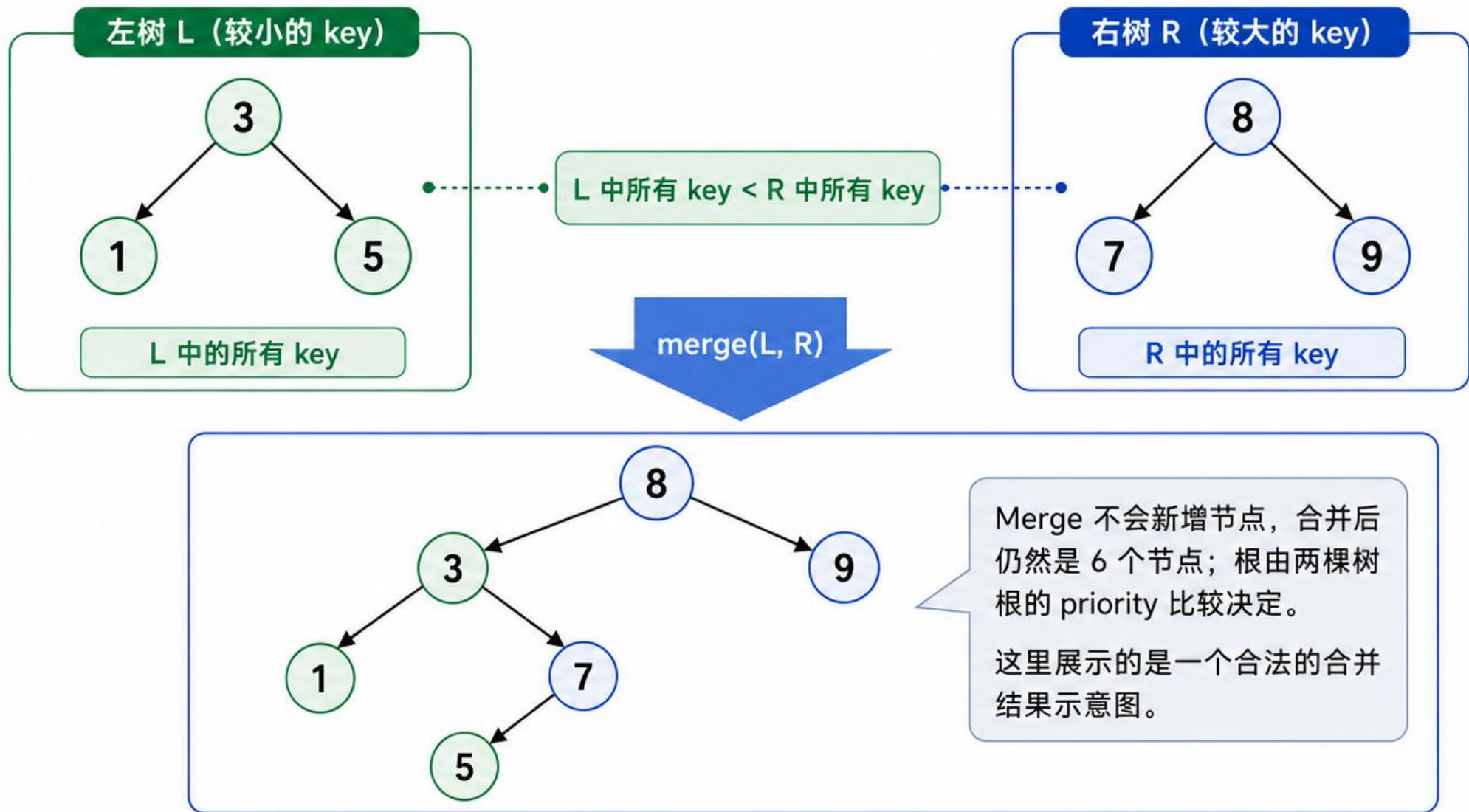


R: > 5



★ 总结: `Split(x, key)` 表示按值 key 将以 x 为根的 Treap 分成两个部分: L ($\leq key$) 和 R ($> key$)。

7 Merge 思想 (重点)



Merge 要求：左树中所有 key < 右树中所有 key。

8

Merge 递归过程 (重点)

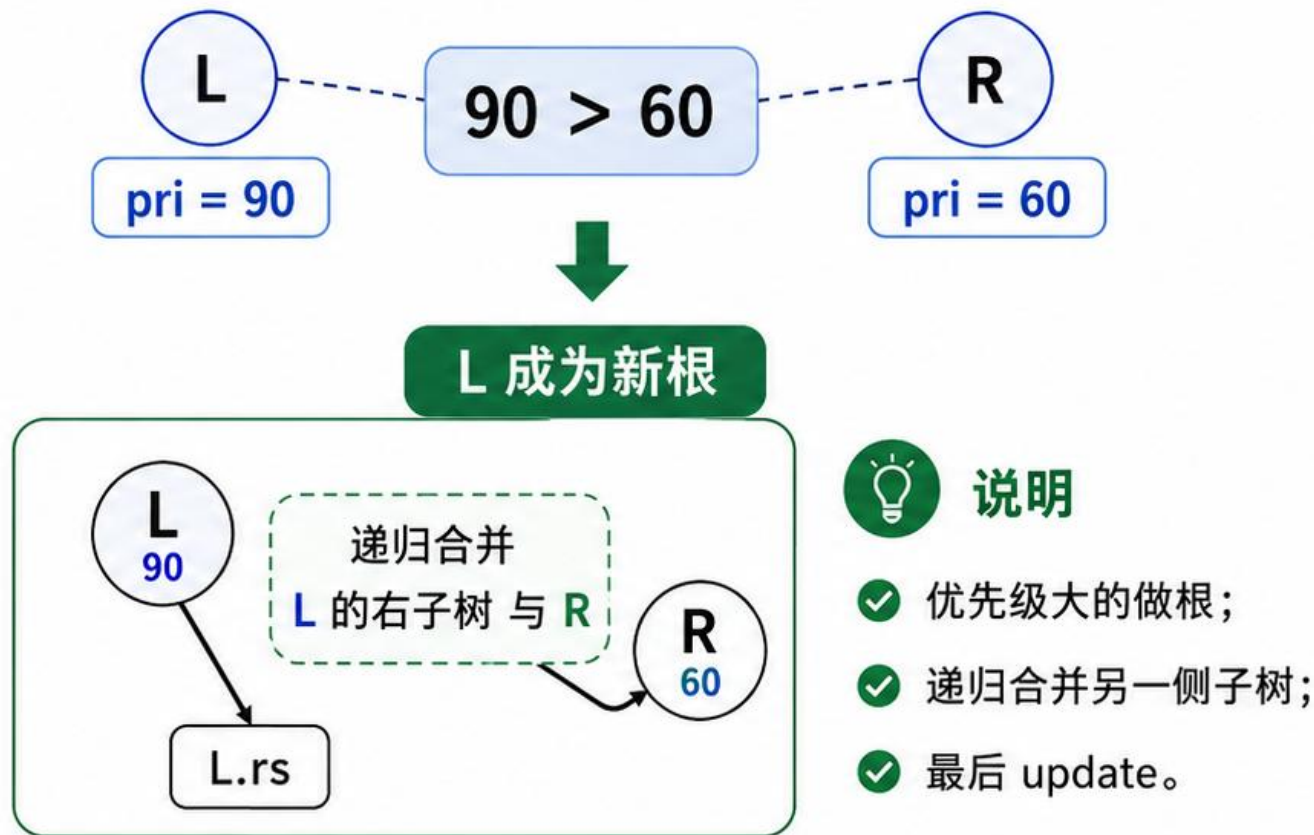


Merge 代码

```
int merge(int L, int R){  
    if(L == 0 || R == 0)  
        return L + R;  
    if(t[L].pri > t[R].pri){  
        t[L].rs = merge(t[L].rs, R);  
        update(L);  
        return L;  
    } else {  
        t[R].ls = merge(L, t[R].ls);  
        update(R);  
        return R;  
    }  
}
```



优先级比较 (示例)

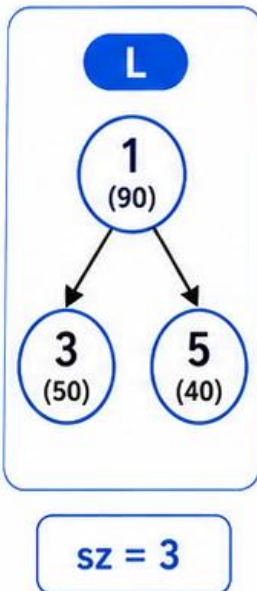


每次 Merge 只决定一个根，然后递归处理剩余部分。

9 Merge 完整过程 (重点)

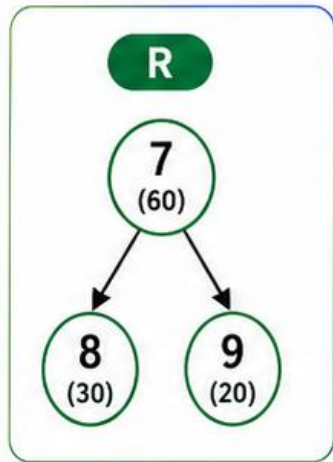
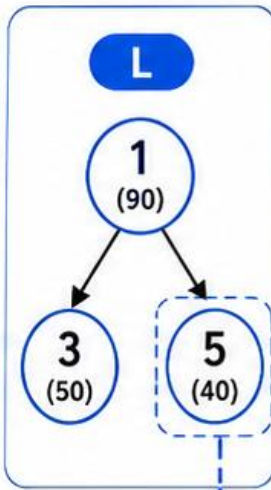
Step 1

比较两棵树根节点的
优先级
 $90 > 60$, L 根获胜



Step 2

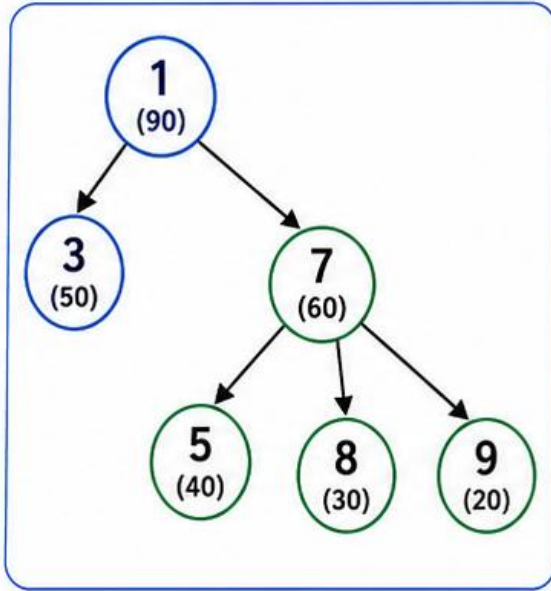
递归合并
L 的右子树与 R



merge(5, R)

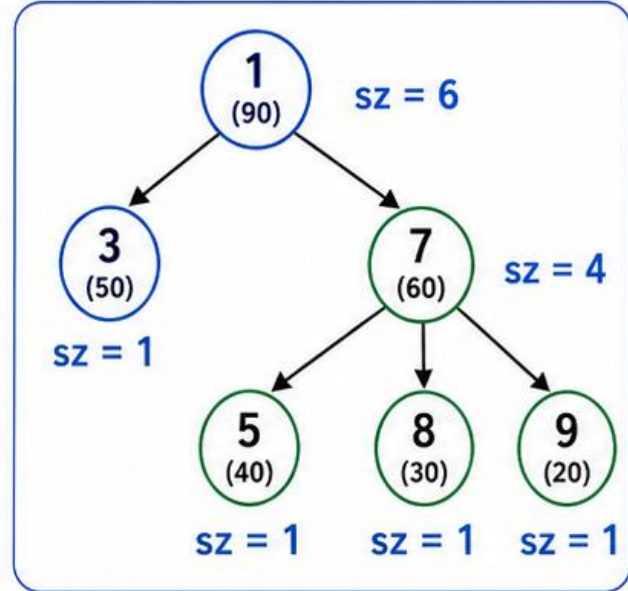
Step 3

R 成为右子树合并结果
(中间状态)



Step 4

更新 sz,
返回整棵树



Merge 的过程: 比较根优先级 → 递归合并子树 → update → 返回。

10 插入操作

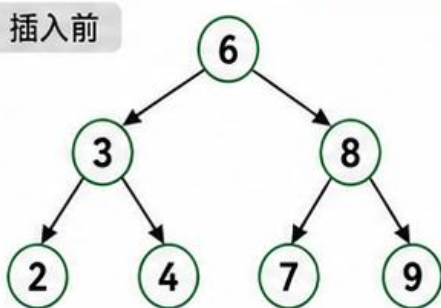
代码 (简化)

```
void insert(int x){  
    int L, R;  
    split(rt, x, L, R);  
    newNode(x);  
    int tmp = merge(L, cnt);  
    rt = merge(tmp, R);  
}
```

● 分裂 (Split) ● 合并 (Merge) ● 辅助/创建

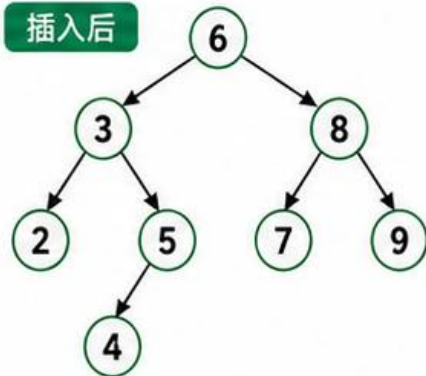
示例：插入 5

插入前



插入 5

插入后



1. 当前 root (根节点 rt)



2. `split(root, x)`: 按 x 分裂



3. 得到两个 Treap:
L (所有键 $\leq x$) 和 R (所有键 $> x$)



4. 创建新节点 x (随机优先级)



5. `merge(L, 新节点)`



6. `merge(结果, R)`



7. 得到新的 root (更新 rt)



插入只需：先 **Split**，再把新节点 **Merge** 回去。

11 删除操作



1. 第一次 Split: `split(rt, x, L, R)`

将树按 x 分裂: L 中所有键 $\leq x$, R 中所有键 $> x$



2. 第二次 Split: `split(L, x-1, L, p)`

在 L 中按 $x-1$ 再分裂: L 中键 $\leq x-1$, p 中键 $= x$



3. 删除目标节点: `p = merge(t[p].ls, t[p].rs)`

p 中的树全是键 $= x$ 的节点, 删除一个:
 p 变为其左右子树的合并 (即删除一个出现位置)



4. 合并回左侧: `merge(L, p)`

将删除后的 p 合并回 L , 得到键 $\leq x$ 的所有节点



5. 最终合并: `rt = merge(merge(L, p), R)`

再与 R 合并, 得到删除后的整棵树



6. 完成: 得到新的根 `rt`

代码 (简化)

```
void erase(int x){  
    int L, R, p;  
    split(rt, x, L, R);  
    split(L, x - 1, L, p);  
    p = merge(t[p].ls, t[p].rs); // 删除一个 x  
    rt = merge(merge(L, p), R);  
}
```

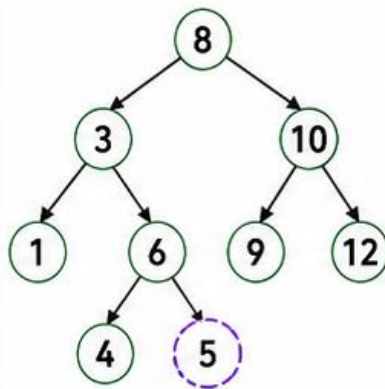
● 分裂 (Split)

● 合并 (Merge)

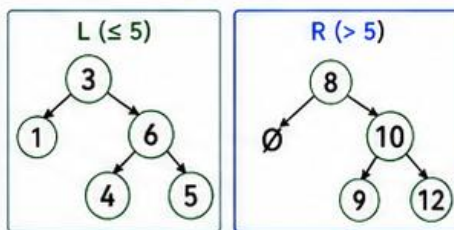
● 删除目标点

示例: 删除键 5

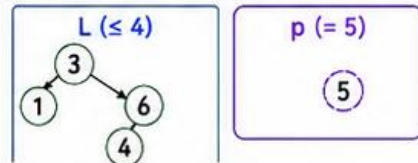
删除前



1) `split(rt, 5, L, R)`

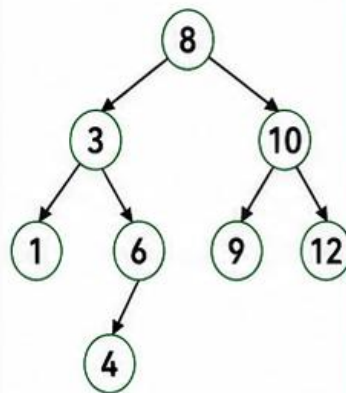


2) `split(L, 4, L, p)`



3) 删除: `p = merge(p.ls, p.rs) → p = ∅`

删除后



删除 = 两次 Split + 删除目标点 + 两次 Merge

12 查询排名 (Rank)



1. 按 $x-1$ 分裂: `split(rt, x-1, L, R)`



2. L 中所有键 $< x$, R 中所有键 $\geq x$



3. 排名 = $t[L].sz + 1$
(即 x 的排名)



4. `merge(L, R)` 还原原树

代码 (简化)

```
void Rank(int x){  
    int L, R;  
    split(rt, x-1, L, R);  
    cout << t[L].sz + 1 << '\n';  
    rt = merge(L, R);  
}
```

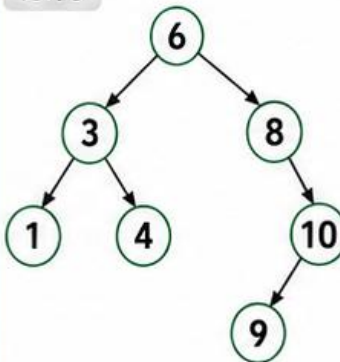
● 分裂 (Split)

● 合并 (Merge)

● 查询/输出

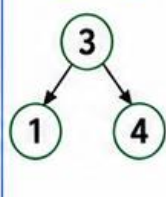
示例: 查询 6 的排名

原树



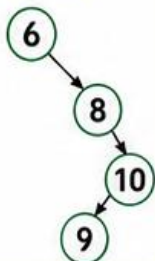
`split(rt, 5, L, R)`

L (< 6)



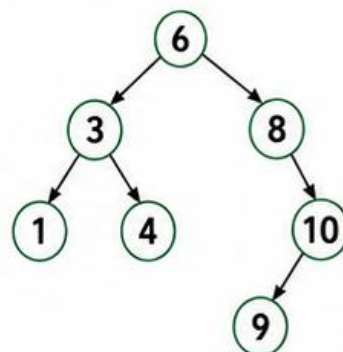
sz = 4

R (≥ 6)



$\text{rank}(6) = 4 + 1 = 5$

`merge(L, R)` 还原



Rank = Split 求左侧大小 + 1, 再 Merge 还原

13 查询第 k 小



思路：利用子树大小 sz 进行递归定位



若 $k = sz(\text{左子树}) + 1$

说明当前节点就是第 k 小，直接返回



若 $k \leq sz(\text{左子树})$

第 k 小在左子树中，递归查询左子树



否则 ($k > sz(\text{左子树}) + 1$)

第 k 小在右子树中，令 $k = k - sz(\text{左子树}) - 1$
递归查询右子树

代码（简化）

```
int kth(int u, int k){  
    if(k == t[t[u].ls].sz + 1) return u;  
    if(k <= t[t[u].ls].sz) return kth(t[u].ls, k);  
    else return kth(t[u].rs, k - t[t[u].ls].sz - 1);  
}
```



命中根（答案）



往左递归



往右递归（ k 调整）

示例：查询第 5 小

树中键（中序序列）：[1, 3, 5, 7, 8, 9] $k = 5 \rightarrow$ 答案应为 8

Step 1

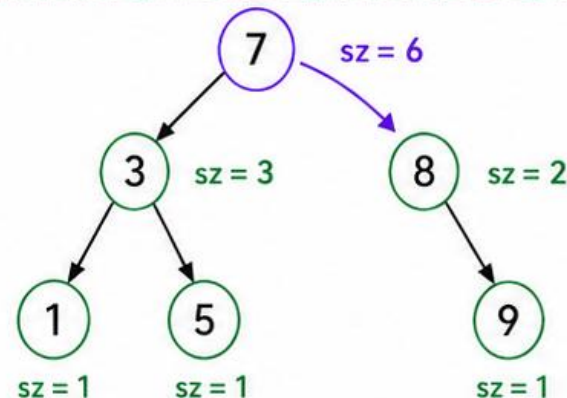
当前节点 $u = 7$

$sz(\text{左子树}) = 3$

比较： $k = 5 > 3 + 1 = 4$

$\rightarrow$ 第 5 小在右子树中

更新 $k = k - 3 - 1 = 1$



Step 2

当前节点 $u = 8$

$sz(\text{左子树}) = 0$

比较： $k = 1 = 0 + 1$

$\rightarrow$ 当前节点就是第 1 小，即答案 8



Step 3

返回：答案节点 = 8（即第 5 小）



第 k 小 = 利用 sz 判断往左、命中根、还是往右

14 前驱与后继



前驱 pre(x)

- 1 分裂: `split(rt, x-1, L, R)`
- 2 答案: 左子树 L 中的最大键
即 `kth(L, t[L].sz)`
- 3 合并: `rt = merge(L, R)`
- 4 返回答案

代码 (前驱)

```
void getpre(int x){  
    int L, R;  
    split(rt, x - 1, L, R);  
    cout << t[kth(L, t[L].sz)].key  
         << '\n';  
    rt = merge(L, R);  
}
```



后继 nxt(x)

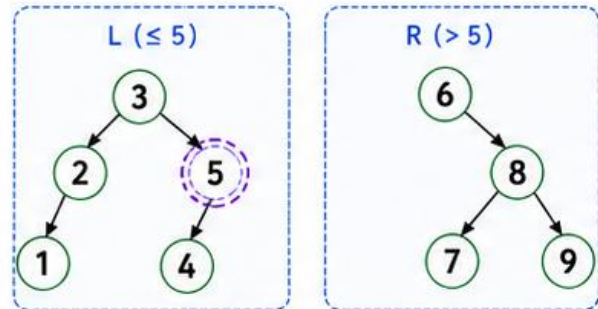
- 1 分裂: `split(rt, x, L, R)`
- 2 答案: 右子树 R 中的最小键
即 `kth(R, 1)`
- 3 合并: `rt = merge(L, R)`
- 4 返回答案

代码 (后继)

```
void getnxt(int x){  
    int L, R;  
    split(rt, x, L, R);  
    cout << t[kth(R, 1)].key << '\n';  
    rt = merge(L, R);  
}
```

查询 6 的前驱

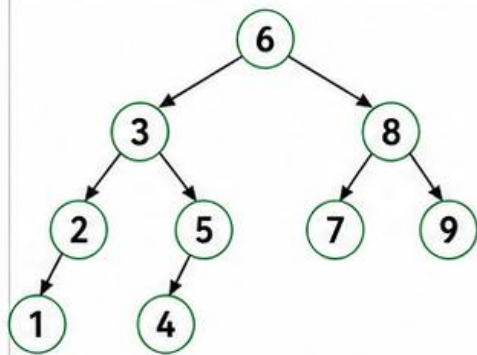
1) `split(rt, 6-1=5, L, R)`



2) 左边最大键
= `kth(L, t[L].sz)`
= 5

3) `merge(L, R)` 恢复原树

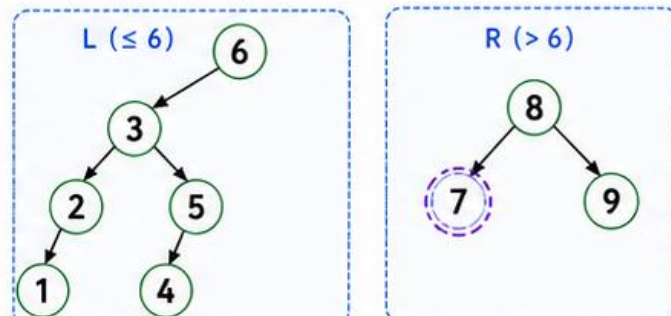
示例: 当前 FHQ Treap



○ 查询键 x □ 分裂后的两部分

查询 6 的后继

1) `split(rt, 6, L, R)`



2) 右边最小键
= `kth(R, 1)`
= 7

3) `merge(L, R)` 恢复原树



前驱看左边最大, 后继看右边最小

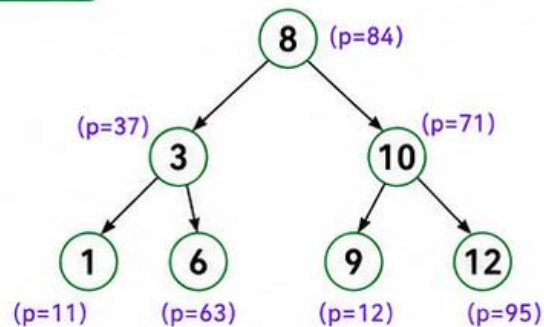
15 时间复杂度与特点

操作复杂度一览（期望值）

操作	说明	期望时间复杂度
 <code>split(rt, x)</code>	按 x 分裂树	$O(\log n)$
 <code>merge(L, R)</code>	合并两棵树	$O(\log n)$
 <code>insert(x)</code>	插入 x	$O(\log n)$
 <code>erase(x)</code>	删除 x	$O(\log n)$
 <code>rank(x)</code>	查询 x 的排名	$O(\log n)$
 <code>kth(k)</code>	查询第 k 小	$O(\log n)$
 <code>pre(x) / nxt(x)</code>	前驱 / 后继	$O(\log n)$

为什么期望为 $O(\log n)$?

- 每个节点的 **priority** 随机独立。
- 在 `merge` 的选择过程中，节点被选作根的概率等于 $1 / \text{子树大小}$ 。
- 因此树高的期望为 $O(\log n)$ ，所有操作沿一条根路径递归完成。



随机 **priority** 使结构期望平衡

FHQ Treap 的特点

- ✓ 无旋转
通过 `split` / `merge` 维护平衡，无需旋转操作。
- ✓ 代码简洁
逻辑清晰，递归结构简单，易于实现与理解。
- ✓ 操作统一
所有操作都可由 `split` 和 `merge` 组合完成。
- ✓ 易扩展
便于维护附加信息（`size`、`sum` 等），支持可持久化 / 区间操作等扩展。



分裂 (Split)



合并 (Merge)



扩展 / 持久化



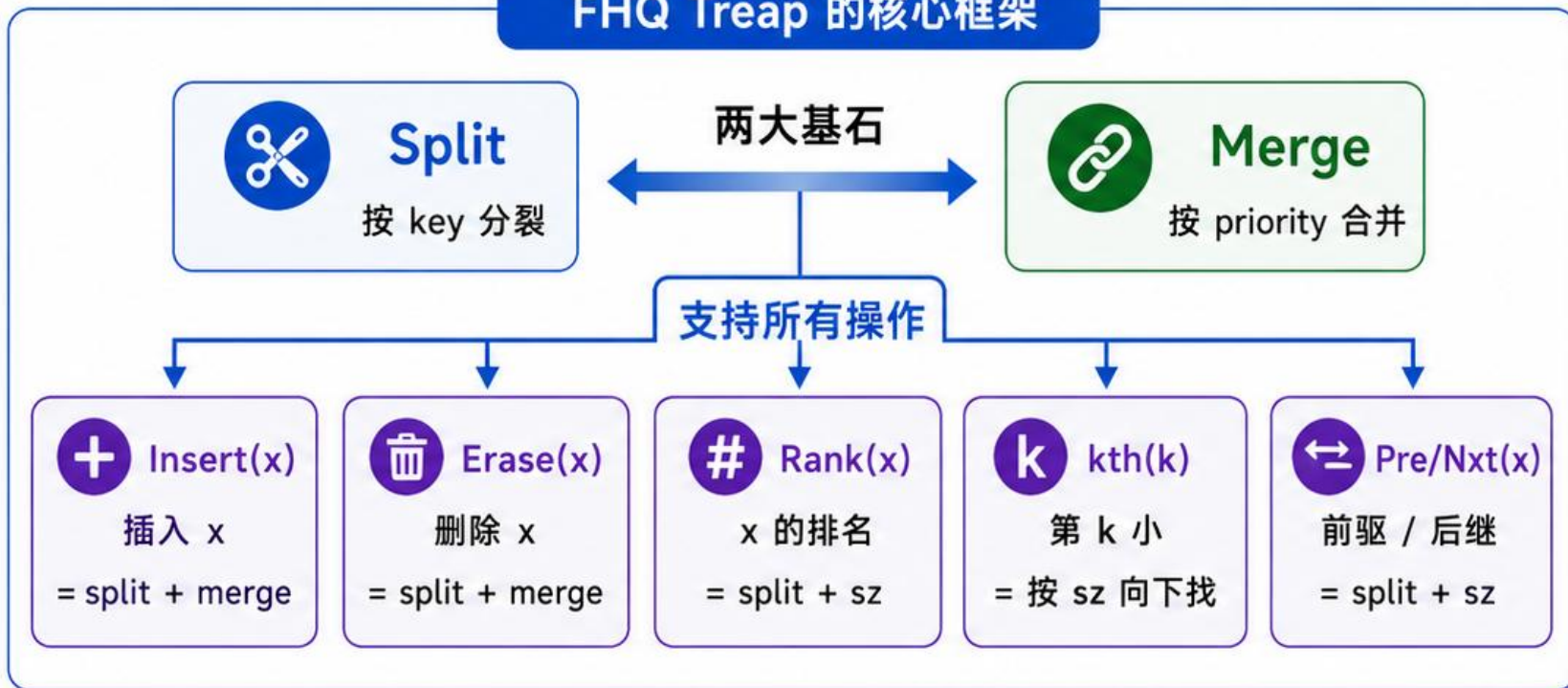
区间操作支持



FHQ Treap 的所有核心操作，期望复杂度均为 $O(\log n)$

16 总结

FHQ Treap 的核心框架



三大关键支撑



key 维护 BST 性质

左子树 $\text{key} \leq \text{根} \leq \text{右子树}$



priority 维护堆性质

父节点 $\text{priority} \geq \text{子节点}$



sz 支持第 k 小与排名

sz = 子树大小 (含自身)



核心回顾

1. FHQ Treap = **BST** + **随机优先级** + **Split/Merge**
2. **Split**: 按 key 分成两棵树;
3. **Merge**: 按 priority 合并;
4. 绝大多数操作 = **Split** + **Merge** 的组合。

学习建议



先熟练 **split / merge**,
再写 **insert / erase / rank / kth**。



掌握 **Split** 和 **Merge**, 就掌握了 **FHQ Treap** 的核心